When Software Fabricates Assurance: Seven Design Principles for Trustworthy Governance, Risk and Compliance Systems

Name of the Track: Track 1 – Information Systems: Theory and Practice (Digital Ethics and Policy: Regulation and Compliance)

Type of Submission: Ongoing research (design science research; artifact and technical evaluation completed, naturalistic evaluation pending)

Abstract

1. Purpose

Boards, auditors and regulators rely on governance, risk and compliance (GRC) information systems for evidence that risks are identified, rated, approved and controlled. Research on these systems has examined their adoption and value. We found no study asking whether the evidence they present can itself be trusted. A code and runtime audit of an existing risk management application found that the software invented risk ratings, discarded approvals, accepted unauthenticated authority and reported failed database writes as successes. This study asks which design failures compromise the integrity of assurance evidence. It also asks which design principles prevent them, and how well an instantiation of those principles performs against the original system.

2. Design/Methodology/Approach

This is a design science study. We read the code of an existing risk management application, ran it, and recorded seven distinct ways in which it failed. Each failure was traced to theory that already explains it, from integrity models, fail-safe defaults and complete mediation to separation of duties, tamper-evident logging and the critique of audit as ritual, and a design principle was drawn from that theory. We built a new system, ERMA-GRC, on these principles. The old and new systems then faced 23 scenarios over the same stored data, under a protocol time-stamped in the project repository beforehand. Eighteen scenarios come from the audit, four go beyond it, and one was meant to fail. All deviations are reported.

3. Findings

Out of the 18 scenarios derived from the audit, the original system prevented the failure in 4 of them. On the other hand, ERMA-GRC prevented all 18 failures. It rejected writes made outside a session, stopped users from approving their own records, stamped each audit line with the signed-in user and not the claimed name, detected an edited entry and reported database unavailability as an error. Beyond the audit, it prevented 3 of the 4 failures, including two simultaneous approval decisions on the same record. Stored results were read from the database stand-in, not from whatever a system claimed. There were two instances where the new design failed, and we document both. While one key risk indicator showed the appropriate state on the screen, a client connecting directly to the server could set any state it liked. Also, a person holding the audit key could change the audit trail offline, undetected after a restart. The principles also depend on one another: separation of duties is only as reliable as the audit trail it reads.

4. Originality/Value

We treat the integrity of assurance evidence as something a GRC system's design can protect or destroy, rather than as an organisational outcome. This paper provides a taxonomy of failures observed in a working system, seven principles together with two testable propositions towards a design theory, and a scenario protocol that others can rerun. Buyers and auditors may use the principles as a checklist. The two design failures define the next three tasks: off-system anchoring of audit trails, computing all metrics on the server and testing the principles with practitioners.

Keywords

assurance integrity; GRC information systems; design science research; enterprise risk management; tamper-evident audit trail; separation of duties